Configuración de un entorno de desarrollo en C++ y primer programa de consola

Malek Cheheid

Laboratorio de Programación (LPR)

Escuela de Educación Superior Técnica N.º 1 «Eduardo Ader» — Vicente López

5° año — 3° Grupo A

Docente: Yamil Ganduglia

https://github.com/malekcheheid-art/holamundo

2026

Objetivo

Configurar un entorno de desarrollo integrado (IDE) en Windows 11, escribir un primer programa de consola en C++, compilarlo con GCC y ejecutarlo desde la terminal. El propósito de la actividad no fue aprender C++ en profundidad, sino comprender el flujo completo de trabajo: cómo se escribe el código, cómo el compilador lo transforma en un ejecutable y cómo se ejecuta ese archivo.

Entorno utilizado

Utilicé Visual Studio Code como editor de texto. Es el entorno con el que ya estaba acostumbrado: lo vengo usando en la escuela desde el año pasado, de modo que la interfaz, los atajos de teclado y la terminal integrada no me resultaron extraños.

El compilador empleado es GCC (TDM-GCC 10.3.0), instalado de forma local en la computadora. La terminal integrada de Visual Studio Code (Ctrl + ñ) fue la herramienta utilizada para compilar y ejecutar el programa.

Tabla 1

Extensiones instaladas en Visual Studio Code

Extensión	Función
C/C++ (Microsoft)	Resaltado de sintaxis e inteligencia de código
C/C++ Extension Pack	Depuración y navegación de código
Code Runner	Ejecución del código con un clic

Conceptos básicos revisados

Código fuente. Conjunto de líneas de texto escrito en un lenguaje que la computadora no comprende directamente, como C++ en este caso.

Compilador. Programa que traduce el código fuente a código máquina o binario, generando un archivo ejecutable (.exe). Sin compilador, el código no produce ningún resultado.

Depurador (debugger). Herramienta que permite ejecutar el programa paso a paso para localizar errores de lógica que no se aprecian a simple vista.

Path. Lista de carpetas en la que el sistema operativo busca los archivos ejecutables. Si g++ no figura en el Path, la terminal no sabe dónde encontrarlo.

Terminal. Ventana de texto plano desde la que se escriben comandos directos al sistema operativo, sin recurrir al mouse.

Lo que me costó configurar

Lo más difícil no fue el compilador ni las extensiones, sino un aspecto más básico: saber en qué carpeta se encontraba la terminal.

La consigna solicita compilar mediante los siguientes comandos:

```
g++ src/main.cpp -o src/holamundo.exe
.\src\holamundo.exe
```

Cuando la terminal se inicia en C:\Users\[nombre de usuario] en lugar de la raíz del proyecto holamundo, el comando no encuentra el archivo src/main.cpp y devuelve el siguiente error:

```
g++.exe: error: src/main.cpp: No such file or directory
```

La solución consiste en cambiar de directorio previamente:

```
cd C:\ruta\al\proyecto\holamundo
pwd   # verifica la ubicación actual
```

El error resultó confuso al principio porque el compilador se encontraba instalado correctamente. El problema no radicaba en el software, sino en la ruta desde la cual se ejecutaba el comando.

Estructura del proyecto

```
holamundo/
|-- .gitignore
|-- README.md
|-- LICENSE
|-- src/
|   `-- main.cpp
|-- docs/
|   `-- InformeProyectoHOLAMUNDO.pdf
`-- capturas/
    |-- gpp_version.png
    |-- extensiones_vscode.png
    `-- ejecucion_hola_mundo.png
```

Código fuente

El archivo src/main.cpp contiene el punto de entrada del programa. La consigna exigía modificar la salida de pantalla con el nombre y apellido del estudiante, a fin de evitar que todos los trabajos fueran idénticos.

```
#include <iostream>

using namespace std;

int main()
{
    cout << "====================================================" << endl;
    cout << "  Hola Mundo! Bienvenido al Taller de LPR 2026.        " << endl;
    cout << "  Estudiante: Malek Cheheid                          " << endl;
    cout << "====================================================" << endl;

    return 0;
}
```

Compilación y ejecución

Desde la terminal de Visual Studio Code, situado en la raíz del proyecto:

```
g++ src/main.cpp -o src/holamundo.exe
.\src\holamundo.exe
```

La salida obtenida fue:

```
==================================================
  Hola Mundo! Bienvenido al Taller de LPR 2026.
  Estudiante: Malek Cheheid
==================================================
```

El código de retorno 0, indicado en return 0;, informa al sistema operativo que el programa finalizó de manera satisfactoria.

Control de versiones

El proyecto se versiona con Git y se aloja en GitHub. El archivo .gitignore excluye del repositorio los elementos que no corresponden al código fuente: la carpeta .vscode/, que contiene la configuración personal del editor, y el ejecutable generado por el compilador (*.exe).

Se commitean únicamente los archivos pertenecientes al proyecto: el código fuente, el README, la licencia y las capturas de evidencia.

Cuestionario técnico de control

1. ¿Qué entorno de desarrollo te parece más intuitivo a primera vista? ¿Por qué?

Visual Studio Code. Principalmente porque ya lo venía usando en la escuela desde el año pasado, de manera que la interfaz y los atajos no me resultaron extraños. Al abrir una herramienta que ya conozco, no se pierde tiempo aprendiendo su funcionamiento y se puede prestar atención a lo que se está programando.

2. ¿Qué características de un IDE creés que te facilitan más el aprendizaje?

La que más me ayudó fue el resaltado de sintaxis y el autocompletado que aporta la extensión C/C++ de Microsoft. Al escribir una función, el editor sugiere los parámetros y el tipo de dato, lo que permite observar cómo se utiliza una función sin necesidad de recurrir a la documentación.

También la terminal integrada. Poder compilar sin salir del editor vuelve rápido el ciclo «escribo, compilo, veo el error», y ver el error de inmediato es lo que permite comprender qué ocurrió en lugar de leer el código e interpretar a ciegas.

3. De las tareas realizadas, ¿cuál te resultó más difícil de configurar?

El paso de compilación del programa, y no por el compilador sino por un detalle de la terminal. Aparecía el error *src/main.cpp: No such file or directory* y lo primero que pensé fue que GCC estaba mal instalado o que faltaba alguna entrada en el Path. El problema real era que la terminal se encontraba en *C:\Users\[nombre de usuario]* y no en la carpeta del proyecto, por lo que la ruta relativa *src/main.cpp* no existía desde allí.

Lo que me costó fue comprender que el error no provenía del compilador, sino del lugar desde el cual se estaba ejecutando el comando.

4. ¿Qué diferencia técnica encontrás entre un «Editor de Texto» y un «IDE»?

Un editor de texto sirve únicamente para escribir y guardar archivos. Ofrece resaltado de sintaxis y nada más: no compila, no ejecuta ni depura. Sublime Text, por ejemplo, es muy rápido para editar archivos sueltos, pero si se le solicita compilar, no puede hacerlo.

Un IDE es un editor de texto al que se suman un compilador y un depurador en una misma ventana. Esta es la diferencia fundamental: el IDE conoce el lenguaje completo y no solo el texto. Eso le permite saber qué es una función, dónde está definida, qué tipo de dato devuelve y, en consecuencia, proponer sugerencias correctas y saltar a la definición con un clic.

Visual Studio Community, además, integra el ciclo completo: se escribe el código, se pulsa F5 y se depuran poniendo breakpoints para observar el valor de cada variable en el momento exacto en que el programa pasa por allí.

5. ¿Qué tipo de aplicaciones sueñan con desarrollar al llegar a 6° y 7° año? ¿Por qué?

Me gustaría desarrollar aplicaciones web. Es el tipo de proyecto en el que el resultado se aprecia con rapidez: se escribe algo, se prueba en el navegador y ya está funcionando. Me resulta motivador ver que lo que escribí en el código aparece en pantalla.

Elegí web porque es el ámbito donde más se programa actualmente y donde existen más posibilidades de encontrar trabajo.

6. ¿Qué otros lenguajes de programación te gustaría investigar además de C++?

Me gustaría conocer JavaScript y Python.

JavaScript me interesa porque acompaña al desarrollo web, que es la línea que me gustaría

seguir. Python porque lo encuentro en ámbitos muy distintos: scripts, automatización de tareas y también desarrollo. Quiero comprobar si se trata de un lenguaje más sencillo de aprender que C++.

7. ¿Qué tan importante considerás que es el orden y la nomenclatura de tus archivos?

Mucho. En esta misma actividad ocurrió: la consigna pide que *main.cpp* se encuentre dentro de *src/*, y el compilador lo busca allí. Si el archivo se denominara *principal.cpp* o estuviera en la raíz, el comando indicado en la consigna no habría funcionado.

Cuando un proyecto crece, la diferencia entre *carpeta final/* y *carpeta final v2/* acaba siendo la diferencia entre encontrar un archivo en diez segundos o perder media hora buscándolo.

8. Si pudieras diseñar el «entorno ideal», ¿qué le agregarías?

Un detector automático de la carpeta de trabajo. Que la terminal supiera siempre dónde me encuentro y me avisara, o mejor aún, que me ubicara automáticamente en la raíz del proyecto sin tener que escribir *cd* de forma manual.

Lo menciono porque en esta actividad el error me detuvo durante un rato. Si el editor me hubiera indicado «estás en C:\Users\Malek, ¿deseas ir a holamundo?», lo habría resuelto en un segundo.

9. ¿Cómo creés que la Inteligencia Artificial puede ayudar a un programador?

Utilizándola para explicar conceptos y mostrar ejemplos, no para que escriba el código en mi lugar. Si le solicito el código completo de un ejercicio, no aprendo a resolverlo. Si le pido que me explique por qué una línea genera error, ahí sí aprendo, porque todavía tengo que comprender el resto del programa.

La diferencia está en el uso que se le dé: si se emplea como quien traduce algo que ya comprendo, resulta útil; si se emplea como quien realiza el trabajo por mí, no lo es.

10. ¿Cuál es tu meta principal para este año en la materia LPR?

Llegar a comprender C++ más allá de la sintaxis. Esta actividad constituyó el primer contacto real con la temática, y me di cuenta de que saber escribir código no equivale a entender cómo funciona internamente.

Mi meta es que, para fines del año, pueda observar un programa y comprender qué realiza cada parte, y no solamente que funcione.